

Property Owner Service

business onboarding and trust-management service

user-service = Who is this person?

property-owner-service = Is this person/business allowed and trusted to operate properties/businesses on the platform?

.

1. What is a PropertyOwner?

In this code, PropertyOwner is not another copy of User.

We already have a user identity:

```
user_account
-----
id
keycloakUserId
email
phone
...
```

PropertyOwner adds the **business-owner profile**:

```
public class PropertyOwner extends AuditableEntity {

    private UUID id;
    private UUID userId;

    private OwnerType ownerType;

    private String displayName;
```

```

private String businessName;
private String businessRegistrationNumber;
private String taxNumber;

private String phoneNumber;
private String email;
private String telegram;

private OwnerVerificationStatus verificationStatus;

private Instant verifiedAt;
private UUID verifiedBy;
private String rejectionReason;

private Boolean active;
}

```

Conceptually:

User

```

|
| "I am Piseth"
|

```

PropertyOwner

```

|
| "I operate Piseth Restaurant Group"
| "Here is my business information"
| "Here are my verification documents"
|

```

Business Trust Profile

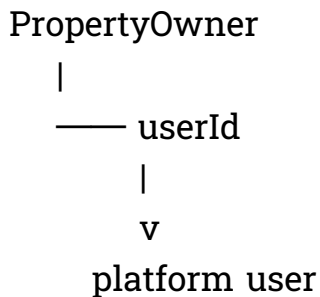
This separation is important in enterprise systems.

user-service owns **identity**.

property-owner-service owns **business ownership and verification**.

2. Why does PropertyOwner contain userId?

This establishes ownership:



For example:

```
User
id = U001
name = Piseth
```

and:

```
PropertyOwner
id = 0001
userId = U001
businessName = "Piseth Restaurant Group"
```

This means:

User U001 owns owner-profile 0001.

That is later used throughout our service.

For example:

```
public Mono<PropertyOwner> getOwnedOwner(
    UUID ownerId,
```

```

        UUID userId) {

    return getById(ownerId)
        .flatMap(owner ->
            owner.getUserId().equals(userId)
                ? Mono.just(owner)
                : Mono.error(
                    new PropertyOwnerAccessDeniedException()));
}

```

This method implements an important authorization rule:

Finding a record and having permission to modify that record are two different things.

We often write:

```
repository.findById(ownerId)
```

and assume that is enough.

It isn't.

User A could send:

```
PUT /property-owners/{owner-of-user-B}
```

So this service checks:

Requested owner

+

Current authenticated user

|

v

Does owner.userId == currentUserId?

Only then is modification allowed.

3. Why only one owner profile per user?

Our validator contains:

```
@Override
public Mono<Void> validateUserHasNoOwner(UUID userId) {

    return propertyOwnerRepository.existsByUserId(userId)
        .flatMap(exists ->
            exists
                ? Mono.error(
                    new PropertyOwnerAlreadyExistsException()
                ) : Mono.empty());
}
```

This is implementing a business rule:

One platform user
|
v
One PropertyOwner profile

Not:

One user
—— Owner Profile 1
—— Owner Profile 2
—— Owner Profile 3
—— Owner Profile 4

Why?

Because PropertyOwner represents the user's **ownership identity**, not each restaurant or KTV.

Later the model can be:

PropertyOwner

|

—— Restaurant A

—— Restaurant B

—— KTV A

—— KTV B

The businesses themselves belong in property-service.

That is a very important service boundary.

4. OwnerType: INDIVIDUAL vs COMPANY

We have:

```
public enum OwnerType {  
    INDIVIDUAL,  
    COMPANY  
}
```

This answers:

What legal/business form is operating the business?

Example:

INDIVIDUAL

Piseth personally operates a restaurant.

versus:

COMPANY

ABC Hospitality Co., Ltd.

operates several restaurants/KTVs.

It does **not** answer:

RESTAURANT or KTV

Those are different dimensions.

We should understand this difference:

OwnerType

INDIVIDUAL

COMPANY

describes **who owns**.

Whereas:

BusinessType

RESTAURANT

KTV

describes **what business is operated**.

Do not combine them into something like:

RESTAURANT_OWNER

KTV_OWNER

inside the owner entity itself.

Those belong more naturally to access-control roles.

5. Why create an owner as DRAFT?

Our factory does:

```
.verificationStatus(  
    OwnerVerificationStatus.DRAFT)  
.active(true)
```

This is a very important business design.

Registration does **not** mean verification.

The workflow starts:

Owner Created

|

v

DRAFT

because the owner may still need to:

- complete business information
- upload documents
- check information
- submit for review

If we created them immediately as:

VERIFIED

then anyone who registered could instantly become a trusted business operator.

That would remove the verification process entirely.

6. Owner verification is a state machine

This is probably the most important concept to demonstrate.

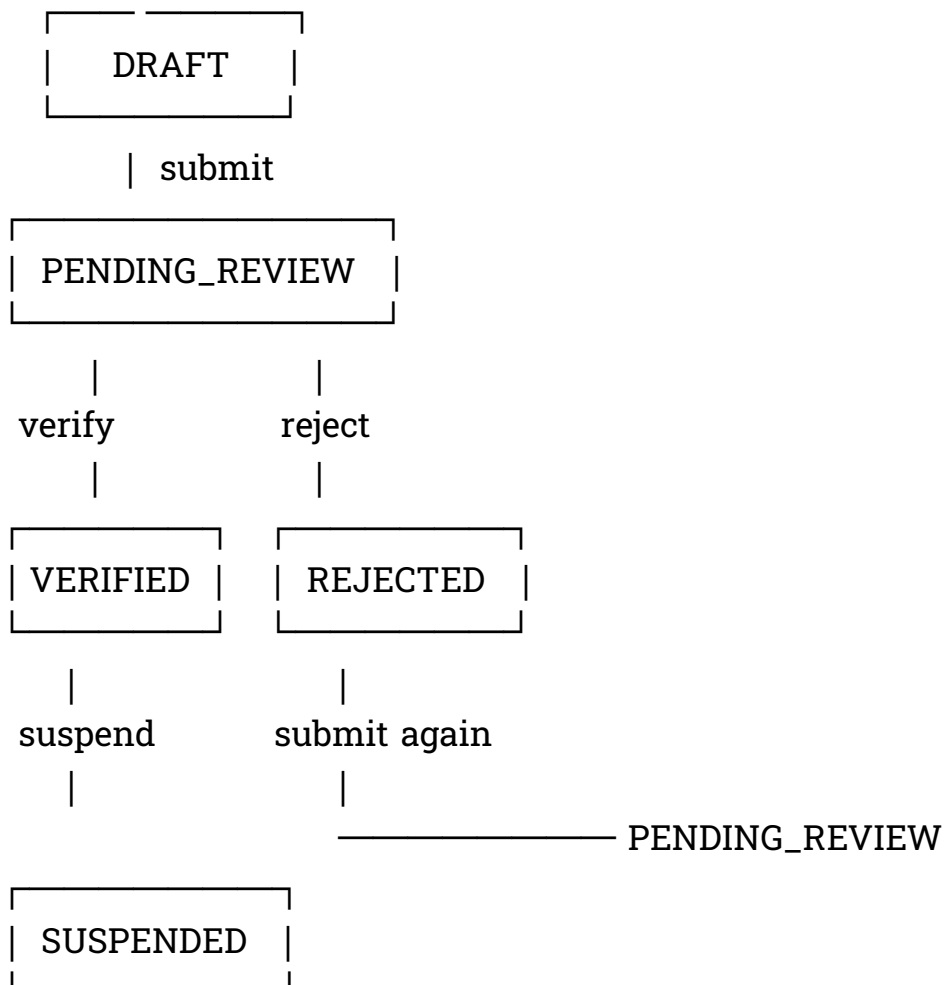
Our statuses are:

DRAFT,
PENDING_REVIEW,
VERIFIED,
REJECTED,
SUSPENDED

We should not think of this as just an enum.

It is effectively a **state machine**.

The normal lifecycle is:



Our code enforces this explicitly.

For example:

```
submit(...)
```

allows:

```
DRAFT => PENDING_REVIEW  
REJECTED => PENDING_REVIEW
```

but not:

```
VERIFIED => PENDING_REVIEW
```

Verification allows:

```
PENDING_REVIEW => VERIFIED
```

Reject allows:

```
PENDING_REVIEW => REJECTED
```

Suspend allows:

```
VERIFIED => SUSPENDED
```

This is much safer than:

```
owner.setVerificationStatus(request.status());
```

because that would allow a client to do:

DRAFT → VERIFIED

without review.

7. Why `changeStatus()` checks allowed previous states

This part is important:

```
if (!Arrays.asList(allowedStatuses)
    .contains(owner.getVerificationStatus())) {

    return Mono.error(
        new InvalidOwnerStatusException(...));
}
```

It protects **business invariants**.

Suppose the current state is:

DRAFT

and someone calls:

PATCH /admin/{id}/verify

Without validation:

DRAFT → VERIFIED

would succeed.

But business logic says:

DRAFT

|

PENDING_REVIEW

|

VERIFIED

Each transition has meaning.

This is why business logic should live in service/facade/domain code rather than only in the frontend.

The frontend can disable buttons, but backend rules remain authoritative.

8. Why do we have OwnerVerificationHistory?

When status changes, we don't only update:

```
owner.setVerificationStatus(target);
```

We also create:

```
OwnerVerificationHistory.builder()  
    .ownerId(owner.getId())  
    .oldStatus(oldStatus)  
    .newStatus(target)  
    .reason(reason)  
    .reviewedBy(reviewedBy)  
    .reviewedAt(Instant.now())  
    .build();
```

This solves an important business problem.

If the owner is currently:

VERIFIED

the current row tells you only:

current status = VERIFIED

It does not tell you:

Who verified it?

Was it rejected before?

Why was it rejected?

How many times did they resubmit?

When did each change happen?

History gives us:

DRAFT -> PENDING_REVIEW

owner submitted

PENDING_REVIEW -> REJECTED

reviewer A

reason: invalid license

REJECTED -> PENDING_REVIEW

owner resubmitted

PENDING_REVIEW -> VERIFIED

reviewer B

This is auditability at the **business level**.

That is different from technical auditing such as:

createdAt
createdBy
updatedAt
updatedBy

Both are useful, but they answer different questions.

9. Technical Audit vs Business History

This is a very good study case comparison.

AuditableEntity answers:

Who created this row?
When?
Who last updated it?
When?

For example:

createdBy = U001
createdAt = 2026-09-05
updatedBy = U900
updatedAt = 2026-09-06

OwnerVerificationHistory answers:

What business event occurred?
What was the old state?
What is the new state?
Who reviewed it?
Why?

Therefore:

Auditing != Business history

We often think one replaces the other. It doesn't.

10. Why documents are separate entities

We have:

```
PropertyOwner
|
—— OwnerDocument
```

instead of putting fields such as:

```
nationalId
passport
businessLicense
taxCertificate
```

directly on property_owner.

That is a good design because an owner can have multiple document types.

For example:

```
PropertyOwner 0001
|
—— NATIONAL_ID
—— BUSINESS_LICENSE
—— TAX_CERTIFICATE
```

A company may require different documents than an individual.

This structure is extensible.

Later we could add:

TOURISM_LICENSE
FOOD_SAFETY_CERTIFICATE
VAT_CERTIFICATE
COMPANY_CERTIFICATE

without modifying the `property_owner` table for every new document type.

11. Why `OwnerDocumentType` is data instead of an enum

Our service has an `owner_document_type` table and seed data.

This is another useful enterprise concept.

We could have written:

```
enum DocumentType {  
    NATIONAL_ID,  
    PASSPORT,  
    BUSINESS_LICENSE  
}
```

But using reference data means document types can have database-managed properties such as:

```
code  
name  
description  
active
```

That allows us to deactivate a type without recompiling code.

For example:

OLD_BUSINESS_CERTIFICATE
active = false

The validator uses:

```
documentTypeRepository  
    .existsByCodeAndActiveTrue(...)
```

So only currently supported document types can be uploaded.

12. Why document uniqueness matters

Our service has:

OwnerDocumentUniquenessValidator

and a database migration:

006-enforce-unique-active-owner-document.yml

This is a strong design choice.

Suppose an owner uploads:

NATIONAL_ID #1
NATIONAL_ID #2
NATIONAL_ID #3

Which one is the real current national ID?

The service prevents multiple active records of the same document type.

Conceptually:

Owner 0001

|
—— NATIONAL_ID active=true (v)
—— NATIONAL_ID active=true (x)

The important lesson is:

Critical business rules should ideally be protected in both application logic and the database.

Application validation gives a nice error.

Database constraints protect against concurrency and programming mistakes.

13. Why document deletion is a soft delete

Our code does:

```
document.setActive(false);  
return repository.save(document);
```

instead of:

```
repository.delete(document);
```

Why?

Documents are part of an important business verification process.

Hard deletion would erase evidence that the record ever existed.

With soft deletion:

```
OwnerDocument  
active = false
```

we keep history and traceability while removing it from normal active use.

This pattern is particularly valuable for:

- compliance records
- verification documents
- payments
- reservations
- business records

14. Why documents also have status

OwnerDocumentStatus is:

PENDING_REVIEW,
VERIFIED,
REJECTED

That means owner verification and document verification are related but not exactly the same thing.

For example:

Owner
PENDING_REVIEW

may have:

National ID	VERIFIED
Business License	PENDING_REVIEW
Tax Certificate	PENDING_REVIEW

Our current code simplifies final approval by doing:

```
verifyPendingDocuments(owner.getId())
```

when the owner is verified.

It changes pending documents to:

VERIFIED

The architectural idea is still useful:

A parent business process can have child verification items.

Later we can make document review more sophisticated without changing the owner model completely.

15. Why Factory is used

Instead of building `PropertyOwner` inside the facade:

```
PropertyOwner.builder()  
    ...  
    .build();
```

we have:

`PropertyOwnerFactory`

with:

```
createDraft(...)
```

This expresses intent.

Compare:

```
factory.createDraft(userId, request)
```

with:

```
new PropertyOwner(...)
```

The first tells the reader:

When an owner is created, it must be initialized according to our DRAFT business rules.

The factory controls defaults:

```
verificationStatus = DRAFT  
active = true
```

Later we might add:

```
onboardingVersion  
riskLevel  
country  
source
```

and every caller does not need to know the initialization rules.

16. Why Facade exists

We flow uses classes like:

```
CreatePropertyOwnerFacade  
UpdatePropertyOwnerFacade  
ChangePropertyOwnerStatusFacade  
OwnerDocumentFacade  
SearchPropertyOwnerFacade
```

The facade represents a **use case**.

For example:

CreatePropertyOwnerFacadeImpl

orchestrates:

Get current user

|

Validate owner doesn't exist

|

Create draft entity

|

Persist

|

Map response

The facade doesn't need to implement every low-level detail.

Instead it coordinates:

CurrentUserReader

Validator

Factory

Writer

Mapper

This supports Single Responsibility.

17. Why Reader / Writer separation exists

We have:

PropertyOwnerReader

PropertyOwnerWriter

This may initially look excessive to us.

Repository APIs are persistence-oriented:

```
findById  
save  
existsBy...
```

But application code often needs domain-oriented operations:

```
findById  
getOwnedOwner  
getByUserId
```

For example:

```
getOwnedOwner(ownerId, userId)
```

is more meaningful than repeating:

```
repository.findById(ownerId)  
  .switchIfEmpty(...)  
  .flatMap(...)
```

everywhere.

The Reader becomes a reusable domain access boundary.

18. Why the search uses a custom query repository

This service does not do:

```
repository.findAll()  
  .filter(...)
```

Instead it has:

```
PropertyOwnerQueryRepository  
R2dbcPropertyOwnerQueryRepository  
PropertyOwnerSearchSqlBuilder
```

The SQL builder creates filters dynamically:

```
if (userId != null) { ... }  
if (ownerType != null) { ... }  
if (status != null) { ... }  
if (active != null) { ... }  
if (keyword != null) { ... }
```

and produces:

```
SELECT *  
FROM property_owner  
WHERE ...  
ORDER BY created_at DESC  
LIMIT :limit  
OFFSET :offset
```

plus:

```
SELECT COUNT(*)  
FROM property_owner  
WHERE ...
```

This is much better for enterprise pagination than:

```
Load every owner  
|  
Filter in Java
```

|
Skip
|
Limit

PostgreSQL performs the filtering and paging where the data lives.

19. Why CurrentUserReader is important

Notice that we create use case does not accept:

```
{  
  "userId": "..."  
}
```

from the client.

Instead:

```
currentUserReader.getCurrentUserId()
```

determines who is creating the profile.

That protects against a serious security bug.

Bad API:

```
{  
  "userId": "another-user-id",  
  "businessName": "..."  
}
```

A malicious user could create an owner profile for someone else.

Better:

JWT

|

Authenticated user

|

CurrentUserReader

|

userId

Identity comes from the security context, not from untrusted request data.

20. active and verificationStatus are not the same thing

Our entity has both:

OwnerVerificationStatus verificationStatus;
Boolean active;

We may ask why both exist.

They solve different problems.

verificationStatus answers:

What is the trust/review state of this owner?

DRAFT

PENDING_REVIEW

VERIFIED

REJECTED

SUSPENDED

active answers:

Is this record currently enabled for normal platform operation?

Example:

verificationStatus = VERIFIED

active = false

could mean:

The owner was successfully verified, but temporarily deactivated their account.

This is different from:

verificationStatus = SUSPENDED

which means the platform has suspended their trusted status.

Keeping these concepts separate gives us more flexibility.

21. One business model decision I would emphasize

PropertyOwner.id

as the **owner/business organization identity**.

Then the architecture becomes:

Platform User

|

| 1:1

v

PropertyOwner / Organization

|

| 1:N

v

Properties

For our Restaurant/KTV course:

User U001

|

v

Owner 0001

|

—— Restaurant P001

—— Restaurant P002

—— KTV P003

Access control can then later say:

userId = U001

role = RESTAURANT_OWNER

scope = ORGANIZATION

scopeId = 0001

This is why the owner ID becomes so important in the integration we were about to implement.

22. The complete business story

Imagine:

Piseth wants to operate
"ABC Restaurant Group"
on the platform.

First:

1. User registers

user-service creates:

User U001

Then:

2. Owner profile created

property-owner-service creates:

Owner 0001

userId = U001

status = DRAFT

Then:

3. Owner uploads documents

National ID

Business License

Tax Certificate

Documents start:

PENDING_REVIEW

Then:

4. Owner submits

Owner changes:

DRAFT

|

PENDING_REVIEW

Then admin reviews.

If incorrect:

PENDING_REVIEW

|

REJECTED

with:

reason = "Business license is unreadable"

The owner fixes it and resubmits:

REJECTED

|

PENDING_REVIEW

Finally:

PENDING_REVIEW

↓

VERIFIED

At that point the owner is trusted to operate businesses on the platform.

That is the **real business purpose** of this service.

The five concepts to remember

These are the most important points:

1. **User is not Owner.** User represents identity; PropertyOwner represents business ownership and trust.
2. **Verification is a workflow/state machine**, not just a status field.
3. **PropertyOwner is the parent business identity** that can own multiple Restaurant/KTV properties.
4. **Authorization checks ownership on the backend**, using the authenticated user rather than trusting userId from requests.
5. **History, documents, soft deletion, DB constraints, Factory/Facade/Reader/Writer exist** because real enterprise business logic is more than CRUD.